TOLEDO PRUDENTE CENTRO UNIVERSITÁRIO
Programação Mobile · Prof. Lenon Fachiano Silva

PROJETO INTEGRADOR

Documentação do Checkpoint 02

Identificação

- **Projeto:** Minha Biblioteca Musical
- **Aluno:** Gustavo Martins Franca
- **Disciplina:** Programação Mobile
- **Repositório:** https://github.com/gustavomf1/biblioteca-mobile
- **Web API utilizada:** backend-music (Java 17, Spring Boot, PostgreSQL), desenvolvida pelo próprio aluno.
- **Data da entrega:** 20/10/2026

1. Evolução do projeto

O que existia no Checkpoint 01

- Aplicação com tela única controlada por useState (telaAtual), com menu inferior próprio.
- Listagem, cadastro e exclusão de músicas apenas em memória (dados locais, perdidos ao fechar o app).
- Componentes Botao, CardMusica e MenuNavegacao.

O que foi implementado no Checkpoint 02

- Navegação com React Navigation (Stack Navigator).
- Integração com a Web API REST usando fetch, async/await, GET (consulta) e POST (cadastro).
- Conversão do JSON da API para o model Musica.
- Estados de carregamento, lista vazia e erros (falha de comunicação e erro do servidor).
- Tela de detalhes que consulta a API por id.
- Tela de cadastro que envia a nova música para a API e atualiza a lista ao voltar.
- Separação entre telas/componentes visuais (screens, components) e comunicação com a API (services).

Funcionalidades alteradas

Checkpoint 01	Checkpoint 02
Lista vinda de um array local	Lista vinda da API (GET /api/songs)
Menu inferior com estado telaAtual	Stack Navigator com parâmetros de rota
Cadastro local (array em memória)	Cadastro enviado à API (POST /api/songs), com validação e tratamento de erros
Exclusão local	Removida desta etapa; voltará com DELETE
Botão "Excluir" no card	Botão "Detalhes" no card

2. Navegação

Telas

Tela	Rota	Descrição
BibliotecaScreen	Biblioteca	Lista as músicas vindas da API (tela inicial)
CadastroScreen	Cadastro	Formulário que envia uma nova música para a API
DetalheScreen	Detalhe	Mostra os dados de uma música, buscada na API
ProjetoScreen	Projeto	Informações sobre o projeto

Fluxo de navegação

```
Biblioteca ──("Detalhes" no card)──────────► Detalhe

Biblioteca ──("Cadastrar música")──────────► Cadastro
Cadastro   ──(salvou: goBack)──────────────► Biblioteca (recarrega)

Biblioteca ──("Sobre o projeto")───────────► Projeto
```

O botão voltar do cabeçalho (ou navigation.goBack()) retorna à tela anterior.

Transferência de dados entre telas

Os parâmetros são tipados em src/navigation/types.ts:

```
export type RootStackParamList = {
  Biblioteca: undefined;
  Cadastro: undefined;
  Detalhe: { id: number };
  Projeto: undefined;
};
```

A BibliotecaScreen envia apenas o id: navigation.navigate('Detalhe', { id }). A DetalheScreen lê route.params.id e consulta a API (GET /api/songs/{id}), garantindo que mostre o dado atual do servidor.

Ao salvar uma música, a CadastroScreen chama navigation.goBack(); a BibliotecaScreen usa useFocusEffect para recarregar a lista da API sempre que volta a ter foco, exibindo o novo registro.

3. Integração com a Web API

- **Endereço base:** http://<host>:8080/api, definido em src/services/api.ts.
- Web / iOS (simulador): localhost. Emulador Android: 10.0.2.2.
- Celular físico: variável EXPO_PUBLIC_API_URL no arquivo .env (modelo em .env.example) com o IP do computador.
- **Recurso/entidade:** songs (música).

Endpoints consumidos

Método	Endpoint	Uso no app	Sucesso
GET	/api/songs	Listar músicas (BibliotecaScreen)	200 + array JSON

Método	Endpoint	Uso no app	Sucesso
GET	/api/songs/{id}	Detalhar música (DetalheScreen)	200 + objeto JSON
POST	/api/songs	Cadastrar música (CadastroScreen)	201 + objeto JSON criado

Formato dos dados recebidos

```
{
  "id": 1,
  "title": "Oceano",
  "artist": "Djavan",
  "album": "Djavan",
  "releaseYear": 1989
}
```

A função converter, em src/services/musicaService.ts, transforma esse JSON no model Musica (titulo, artista, album, anoLancamento).

Corpo enviado no POST (Content-Type: application/json), sem id:

```
{
  "title": "Oceano",
  "artist": "Djavan",
  "album": "Djavan",
  "releaseYear": 1989
}
```

Formato de erro do servidor

```
{
  "timestamp": "2026-10-20T18:12:14Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed",
  "details": ["title: title is required"]
}
```

O app exibe o campo message e, quando houver, os itens de details.

Organização do código

```
src/
  components/   Botao, CardMusica, MensagemEstado  (visual)
  screens/      Biblioteca, Cadastro, Detalhe, Projeto (Screen)
  navigation/   types.ts
  services/     api.ts, musicaService.ts  (comunicação com a API)
  models/       Musica.ts
```

4. Tratamento de estado

Cada tela que consulta a API usa três estados com useState:

Estado	Função
musicas / musica	Armazena os dados convertidos recebidos da API
carregando	true enquanto aguarda a resposta
erro	Mensagem a exibir quando a requisição falha

Na CadastroScreen, o estado enviando bloqueia envios repetidos e troca o texto do botão para "Salvando..."; os erros de validação local e os erros da API aparecem no estado erro.

A função carregar() (async/await) liga carregando, limpa erro, chama o serviço dentro de try/catch/finally e desliga carregando ao final. Ela é chamada quando a tela abre: na DetalheScreen por um useEffect (refeito se o id mudar) e na BibliotecaScreen por useFocusEffect, que também roda ao voltar do cadastro. Também é chamada pelos botões "Tentar novamente" e "Atualizar".

Cenários tratados

Cenário	Como aparece para o usuário
API disponível com dados	Lista de cards e contador
API disponível sem registros	"Nenhuma música cadastrada" com botão "Atualizar"
Aguardando resposta	Indicador de carregamento e "Carregando músicas..."
Falha na comunicação	"Não foi possível carregar" + "Não foi possível conectar ao servidor. Verifique sua conexão." + botão "Tentar novamente"
Erro enviado pelo servidor	"Não foi possível carregar" + mensagem (message) devolvida pela API + botão "Tentar novamente"

A falha de comunicação é identificada quando o fetch lança exceção; o erro do servidor, quando response.ok é false. Ambos viram um ErroApi com tipo "rede" ou "servidor".

Como executar

- Subir o banco e a API: docker compose up -d e mvn spring-boot:run (pasta backend-music).
- No app: npm install e npm start.
- Em celular físico, copiar .env.example para .env e ajustar o IP do computador.

Tecnologias e conceitos

- React Native com Expo e TypeScript.
- React Navigation (native stack).
- fetch, async/await, useState, useEffect, useFocusEffect.
- FlatList, ActivityIndicator, Pressable, StyleSheet e Flexbox.

Capturas de tela

Capturas do aplicativo em execução, consumindo a Web API:

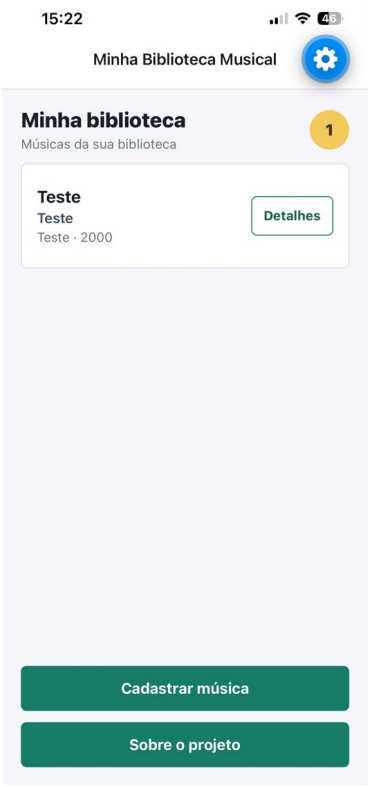


Figura 1 - Tela Minha Biblioteca (lista com dados)



Figura 2 - Tela Cadastrar música



Figura 3 - Tela Detalhes da música